obs-skills 技能互调机制与架构深度分析

> 面试讲解参考文档：聚焦技能间调用机制、tool-calling 模式、simple-harness 编排架构

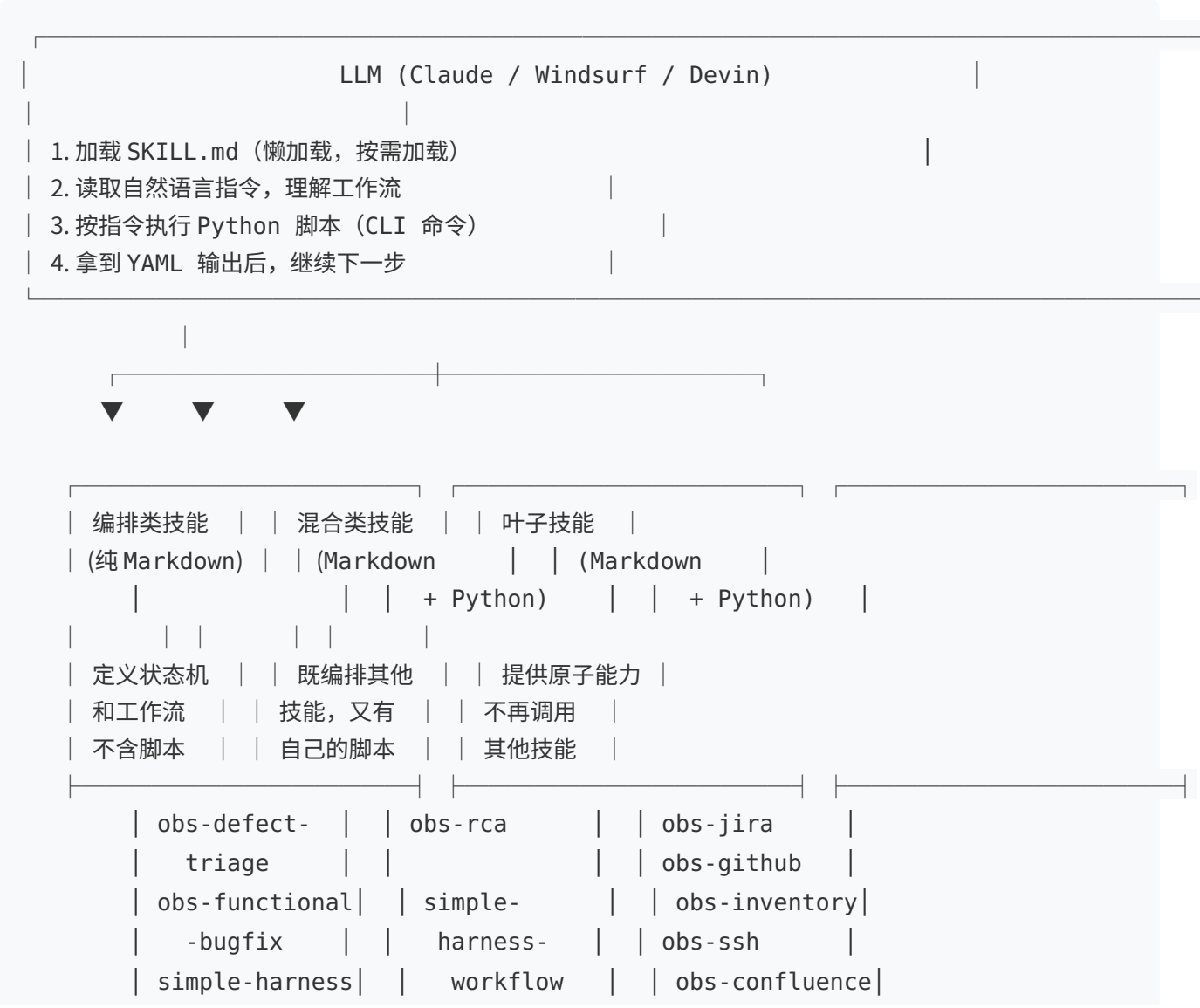
一、核心结论：Prompt-as-Orchestrator 架构

obs-skills 的技能互调**不依赖编程级别的调度框架**——没有 LangChain、没有 CrewAI、没有任何 `skill_A.invoke(skill_B)` 的代码。所有编排发生在 LLM 层面：

- **SKILL.md 就是给 LLM 的操作手册**，用自然语言描述"何时调用什么技能、传什么参数、怎么处理返回值"
- **LLM 读了 SKILL.md 之后自己决定调用顺序**，就像人类工程师读 runbook 一样
- **Python 脚本提供具体能力**（读 JIRA、SSH 到节点、查 GitHub），但脚本之间不互相调用

这意味着：整个 96 技能平台的编排逻辑，本质上是一套**结构化的 prompt 工程**。

二、三层技能架构



```
        |   -orchestrator| |              |   | obs-slack    |
    └──────────────────────┘  └──────────────┘   └──────────────┘
```

2.1 编排类技能（Orchestrator Skills）

纯 SKILL.md，没有 `scripts/` 目录。它们是工作流的"大脑"，通过自然语言指令告诉 LLM 每一步该做什么。

代表：obs-defect-triage——6 步状态机：

```
Defect Understood → Severity Validated → Ownership Confirmed
→ Investigation Complete → Disposition Decided → Disposition Applied
```

在每一步中，SKILL.md 会写：

```
Invoke skill: obs-jira
  → issue-by-key <DEFECT_KEY> --all-comments

Invoke skill: obs-rca
  → quick assessment focused on known-pattern matching

Invoke skill: obs-scrum
  → look up component owner for the defect's component
```

LLM 读到 `Invoke skill:` 就知道要去加载对应技能并执行。

2.2 混合类技能（Hybrid Skills）

既有工作流定义（编排其他技能），又有自己的 Python 脚本。

代表：obs-rca——自身有 5 个 Python 脚本（trace_parser.py、code_lookup.py、deep_trace.py、log_collector.py、correlator.py），同时也编排 obs-inventory、obs-jira、obs-github 和 case skills。

2.3 叶子技能（Leaf Skills）

提供具体的 Python CLI 工具，SKILL.md 里写的是"怎么用我的脚本"，不涉及调用其他技能。

代表：obs-jira——提供 `jira_read.py`（只读）和 `jira_update.py`（写入），遵循读写分离模式。SKILL.md 提供完整的命令语法：

```
uv run scripts/jira_read.py issue-by-key OBS-1234 --all-comments
uv run scripts/jira_update.py add-comment OBS-1234 "RCA complete, root cause is..."
```

三、技能互调的四种模式

模式 1：显式调用指令（Explicit Invocation）

SKILL.md 直接告诉 LLM 调用哪个技能、传什么参数。这是最常见的模式。

```
## Skills Invoked

| Skill | Workflow role |
|-------|---------------|
| **obs-jira** | Read defect details, update status |
| **obs-rca** | Root cause analysis (Phase 0→4) |
| **obs-github** | Submit PR, update description |
```

在工作流步骤中：

```
Step 3: Invoke skill: obs-rca
  → Input: JIRA key + stack trace
  → Phase 0-4: full RCA pipeline
  → Output: ${RCA_DIR}/report.md
```

使用该模式的技能：obs-defect-triage（调用 6 个技能）、obs-functional-bugfix（调用 8+ 个技能）、obs-sc-upgrade-triage、obs-safer-pr 等。

模式 2：条件委托（Conditional Delegation）

obs-rca 的 Phase 2 根据症状关键词匹配，自动委托给对应的 case skill：

```
### Case skills (subcategory: case)

| Skill | Symptom keywords |
|-------|-----------------|
| **obs-dt-unready** | DT unready, stop serving, initialization failure |
| **diagnose-obcc-marker** | OBCC lag, geo-replication stuck, journal region copy |

> 当新 case skill 被添加（subcategory: case），
> 它自动进入 Phase 2 的匹配池，无需修改 obs-rca。
```

这是一个开放式扩展点——加新 case skill 不需要改任何已有代码。类似于插件注册机制，但注册行为通过 YAML front-matter 中的 `subcategory: case` 标签完成。

模式 3：流水线路由（Pipeline Routing）

simple-harness-orchestrator 根据用户当前状态（有什么、在哪一步）做路由决策：

```
用户有什么?
  │
  ├─ (A) 模糊想法
  │     → invoke: simple-harness-design-generator
  │     → then: simple-harness-impact-analyzer
  │     → then: simple-harness-workflow
  │
  ├─ (B) 设计文档
  │     → invoke: simple-harness-impact-analyzer
  │     → if 跨组件: simple-harness-contract-divider
  │     → then: simple-harness-workflow
  │
  ├─ (C) 任务列表，准备开发
  │     → invoke: simple-harness-workflow
  │
  ├─ (D) 恢复之前的工作
  │   → 读 claude-progress.md + feature_list.json
  │   → 定位到对应阶段的技能
  │
  └─ (E) 问题模糊，需要调研
        → invoke: simple-harness-researcher
        → then: simple-harness-design-generator
        → then: simple-harness-impact-analyzer
        → then: simple-harness-workflow
```

关键设计：每次技能完成后，orchestrator 定义了 "Mid-Process Transitions"——告诉 LLM 下一步该路由到哪：

```
### After simple-harness-design-generator completes:
> "Design is approved. Proceeding to impact analysis."
> Invoke: simple-harness-impact-analyzer with the approved design

### After simple-harness-impact-analyzer completes:
> "Spec diff approved. Ready to start implementation."
> Invoke: simple-harness-workflow
```

模式 4：加权选择（Weighted Selection）

当多个技能都能满足请求时，skill-scorer 提供排名依据：

```
weight = avg_rating + log2(1 + usage_count) * 0.5
```

LLM 的决策流程：

```
用户请求
  │
```

```
├── 只有 1 个技能匹配? → 直接使用
│
└── 多个技能匹配?
  → 运行: scorer.py list --format json
  → 选择 weight 最高的技能
   │
   ├── 执行技能
   ├── 运行: scorer.py record <skill-name>      ← 记录使用
   └── 询问用户: "How useful? (1-5)"            ← 收集反馈
     └── 运行: scorer.py rate <skill-name> N
```

形成**正反馈循环**：好技能被用得更多 → 评分更高 → 被选择更多 → 差技能自然下沉。

四、完整调用关系图

4.1 缺陷处理工作流（Defect Workflow Chain）

```
obs-defect-triage
 │
  ├──→ obs-jira .................. 读取缺陷详情、更新状态
  ├──→ obs-confluence ............ 查阅缺陷流程规则
  ├──→ obs-scrum ................. 查找组件归属团队
  ├──→ obs-inventory ............. 解析集群上下文
  ├──→ obs-log-analysis .......... 识别涉及的服务
  ├──→ obs-rca ................... 快速根因分析
 │    │
  │       ├──→ obs-inventory ...... 解析集群信息
  │       ├──→ obs-jira ........... 读 JIRA 详情（条件性）
  │       ├──→ obs-github ......... 代码搜索和查看
  │       ├──→ obs-dt-unready ..... case 委托（DT 故障）
  │       └──→ diagnose-obcc-marker  case 委托（OBCC 延迟）
 │
  └──handoff──→ obs-functional-bugfix
        │
                    ├──→ obs-jira ........... 读取/更新缺陷
                    ├──→ obs-rca ............ 完整 RCA（Phase 0-4）
                    ├──→ obs-github ......... 提交 PR
                    ├──→ obs-storage-remote-ut  远程单元测试
                    ├──→ gradle-build ....... 构建制品
                    ├──→ patch-container .... 热补丁到环境
                    ├──→ obs-ssh ............ 远程验证命令
                    ├──→ obs-log-analysis ... 日志验证修复
                    └──→ obs-confluence ..... 流程规则参考
```

4.2 Simple-Harness 开发流水线

```
simple-harness-orchestrator (入口/路由)
 |
 ├──→ simple-harness-creator (Phase 0: 脚手架)
 |    → 复制 16 个模板文件到项目根目录
 |    → 定制化 AGENTS.md, init.sh, feature_list.json
 |
 ├──→ simple-harness-researcher (Phase 0.5: 调研, 可选)
 |    → 调研行业最佳实践和技术选型
 |    → 输出: research-brief.md
 |
 ├──→ simple-harness-design-generator (Phase 1: 设计)
 |    → 基于代码库产出设计文档
 |    → 与用户迭代直到审批通过
 |
 ├──→ simple-harness-impact-analyzer (Phase 2: 影响分析)
 |    → 扫描代码库，产出 spec-diff.md + feature_list.json
 |    → 大项目: 拆分为多个 batch
 |
 ├──→ simple-harness-contract-divider (Phase 2.5: 合约, 条件性)
 |    → 提取跨组件接口合约: contracts.md
 |    → 构建依赖图: dependency-graph.md
 |    → 分配执行 wave（并行机会）
 |
 └──→ simple-harness-workflow (Phase 3: 实现循环)
      |
           ├─ Planner: 读 spec-diff + contracts → 产出执行计划
      |
           ├─ Generator Coordinator: 分派任务
           |     ├─ Structure Agent: types/schemas/models
           |     ├─ API Agent: routes/handlers/middleware
           |     ├─ Test Agent: tests/fixtures/mocks
           |     ├─ Telemetry Agent: 可观测性 (可选)
           |     ├─ Upgrade Agent: 迁移/兼容 (可选)
           |     └─ Infra Agent: 构建/部署 (可选)
      |
           ├─ Evaluator: 8 维评分 + 规则检查 + 样式检查
           |     → Accept: 标记 passing，下一个 feature
           |     → Revise: 退回 Generator（最多 3 轮）
           |     → Block: 暂停，升级给人类
      |
      └── 多 batch: 完成后路由回 impact-analyzer
```

4.3 基础设施/诊断类互调

```
 obs-sc-upgrade-triage ──→ obs-my-tokens, obs-inventory, obs-ssh, obs-jenkins
obs-s3-throttler ──→ obs-ssh, obs-my-tokens, obs-inventory, obs-cluster-ops
obs-os-install ──→ obs-ssh, obs-inventory, obs-jenkins
obs-safer-pr ──→ obs-github, obs-jira
obs-tier2-workflow-pr ──→ obs-github
obs-cherry-pick ──→ obs-jira (可选)
obs-replication-performance ──→ obs-collect-service-io (可选)
```

五、Simple-Harness 的治理体系

Simple-Harness 不只是一条流水线，它还有一套**宪法式治理机制**，防止 AI Agent 偏离轨道。

5.1 Constitutional Invariants（不可违反的硬规则）

定义在 `harness-constitution.yaml` 中：

规则 ID	内容	作用域
HSS-AUTH-001	没有 Agent 可以批准自己的输出	evaluator, generator, planner
HSS-AUTH-002	Generator 严格按计划实现，不做计划外变更	generator, sub-agents
HSS-SCOPE-001	spec-diff 是实现的上限	generator, planner
HSS-SCOPE-002	Sub-Agent 只能操作声明范围内的文件	structure/api/test agent
HSS-EVIDENCE-001	没有验证证据就不能 Accept	evaluator
HSS-EVIDENCE-002	缺少测试 = 自动 Revise	evaluator
HSS-DRIFT-001	Generator 不做架构决策	generator
HSS-DRIFT-002	Evaluator 不修代码	evaluator
HSS-PROHIB-001	不能在生成的代码中包含密钥	所有
HSS-PROHIB-002	不能在生成过程中修改 harness 文件	generator, sub-agents

5.2 Enforcement Gates（自动检查点）

Gate	触发时机	检查内容
GATE-PRE-GEN	Generator 启动前	计划存在、文件路径真实、无边界重叠、合约已定义
GATE-POST-SUBAGENT	每个 Sub-Agent 完成后	

Gate	触发时机	检查内容
		只修改了边界内文件、无密钥引入、diff 合理、可追溯到计划
GATE-PRE-EVAL	Evaluator 启动前	有实现证据、测试存在、无 lint 错误、init.sh 通过
GATE-POST-EVAL	Evaluator 完成后	所有维度已评分、判决一致、证据非空、样式已检查

5.3 设计灵感来源

参考架构	借鉴了什么
isg-ai-governance (16 Agent 流水线)	Constitutional invariants、Agent 身份与权限声明、行为合约、Unauthorized scope verdict
ruflo (100+ Agent 编排框架)	Enforcement gates、Run ledger（审计日志）、层级拓扑（Queen/Worker 模型）、反漂移检查点

六、与主流 AI Agent 框架的对比

维度	obs-skills (Prompt-as-Orchestrator)	LangChain / CrewAI	AutoGen
编排层	SKILL.md（自然语言）	Python 代码	Python 代码
技能定义	Markdown + YAML front-matter	Python 类/函数	Python 函数
技能发现	`npx openskills install` + category 标签	代码注册	代码注册
跨 IDE	Devin CLI / Windsurf / Claude Code 通用	绑定 Python 运行时	绑定 Python 运行时
贡献门槛	写 SKILL.md + Python 脚本，无需学框架	学习框架 API	学习框架 API
供应商锁定	遵循 Anthropic Agent Skills 开放规范	绑定 LangChain 生态	绑定 Microsoft 生态
状态管理	文件系统（feature_list.json, progress.md）	内存 / 数据库	内存 / 数据库
治理	harness-constitution.yaml + enforcement gates	自行实现	自行实现
代价	依赖 LLM 的指令遵循能力	学习曲线高，灵活性低	多 Agent 对话开销大

七、面试讲解话术

30 秒概述

> "我主导设计了一套 96 技能的 AI Agent 平台。技能之间的调用不靠编程框架，靠 SKILL.md 里的自然语言指令——LLM 读了之后自己决定何时调用什么，就像工程师读操作手册一样。这套架构让任何工程师都能贡献新技能——只要写 Markdown + Python 脚本，不需要学任何 Agent 框架。"

如果问 "技能之间怎么调用？"

> "四种模式。最常见的是**显式调用**——SKILL.md 里直接写 `Invoke skill: obs-jira → issue-by-key KEY`，LLM 读到就执行。第二种是**条件委托**——obs-rca 的 Phase 2 根据症状关键词自动匹配 case skill，新加的 case skill 通过标签注册，无需改 obs-rca 代码。第三种是**流水线路由**——orchestrator 根据用户状态决定路由到 7 个技能中的哪一个。第四种是**加权选择**——当多个技能都能满足请求时，skill-scorer 提供 `weight = avg_rating + log2(usage) * 0.5` 排名，LLM 选权重最高的。"

如果问 "为什么不用 LangChain？"

> "三个原因。第一，**灵活性**——SKILL.md 用自然语言描述条件逻辑、状态机和复杂分支，比 JSON schema 更表达力强。第二，**零耦合**——技能之间没有代码依赖，新增技能不需要改已有代码，通过标签自动发现。第三，**人机同构**——SKILL.md 既是 prompt 也是 runbook 也是脚本文档，三合一降低维护成本。代价是依赖 LLM 的指令遵循能力，所以我们加了防卡死规则（每步时间预算、one-retry rule）和 enforcement gates。"

如果问 "simple-harness 是什么？"

> "这是平台里最复杂的编排系统——7 个技能组成的 AI 辅助开发流水线，从模糊想法到代码交付全覆盖。核心理念是 'constrain before you code'——调研收窄设计空间，设计收窄影响范围，影响分析收窄实现范围，实现受 evaluator 约束。
>
> 最值得一提的是治理体系：我们定义了 10+ 条 constitutional invariants（如'没有测试就不能 Accept'、'Generator 不能修改 harness 文件'），配合 4 个 enforcement gates 在关键节点自动检查。这借鉴了 isg-ai-governance 的宪法式约束和 ruflo 的反漂移机制。"

如果问 "LLM 不会搞错调用顺序吗？"

> "三层防护。第一，**显式指令格式**——统一用 `Invoke skill: X → 参数` 格式，LLM 容易识别和执行。第二，**状态机定义**——工作流技能定义了明确的状态转换（如 `Defect Understood → Severity Validated → Ownership Confirmed`），LLM 沿状态走就行。第三，**防卡死规则**——每步有时间预算（如 Phase 3 最多 15 分钟），one-retry rule（失败只重试一次），no-backtracking rule（前进不回头），保证工作流不会无限循环。"

如果问 "这个平台的技术亮点？"

> "我认为有三个。第一，**开放式扩展**——新 case skill 通过 `subcategory: case` 标签自动注册到 obs-rca 的匹配池，类似于插件机制但零代码。第二，**正反馈 skill-scorer**——使用和评分形成闭环，好工具自然上浮差工具下沉，类似推荐系统的协同过滤思路。第三，**Constitutional governance**——

> 受宪法启发的不可违反规则 + enforcement gates，在 AI 生成代码的场景下提供了可验证的安全边界。"

附录：关键文件索引

文件	作用
`AGENTS.md`（项目根）	全局规范：技能格式、PEP 723、skill-scorer 协议、VDOM 约定
`categories.json`	11 个分类 + 19 个子分类的技能分类体系
`skills/skill-scorer/`	加权排序引擎：usage + rating → weight
`skills/obs-rca/SKILL.md`	4 阶段 RCA 流水线，编排 4+ 技能，条件委托 case skills
`skills/obs-defect-triage/SKILL.md`	6 步状态机，编排 6 个技能，handoff 到 bugfix
`skills/obs-functional-bugfix/SKILL.md`	端到端修复流程，编排 8+ 个技能
`skills/simple-harness-orchestrator/SKILL.md`	开发流水线入口，路由到 6 个 harness 技能
`skills/simple-harness-workflow/SKILL.md`	Planner/Coordinator/Generator/Evaluator 循环
`docs/simple-harness-v2-report.md`	V2 架构设计报告，含参考架构对比
`docs/vdom-convention.md`	HTML 内容的 VDOM 表示约定
`docs/daily-dump-convention.md`	批量数据导出约定